

Task 21: Redux Toolkit

Summary

In this task, I learned about Redux Toolkit (RTK), the official and recommended way to write Redux logic in modern React applications. Redux Toolkit simplifies state management by reducing boilerplate code and providing built-in utilities for creating stores, reducers, and actions. It makes Redux easier to learn, write, and maintain compared to the traditional Redux approach.

I learned how to use `configureStore()` to create the Redux store, `createSlice()` to define state, reducers, and actions in a single place, and how actions automatically update the application state. I also understood how React components interact with Redux using the `useSelector()` Hook to read data from the store and the `useDispatch()` Hook to dispatch actions and update the state.

Additionally, I explored the benefits of centralized state management, where multiple components can access and update shared data without excessive prop passing. Redux Toolkit improves application scalability, code organization, and maintainability, making it suitable for managing complex state in large React applications.

Learning Outcome

After completing this task, I understood how to use Redux Toolkit to create and manage a centralized application state, configure the Redux store, create slices, dispatch actions, and access state using React Redux Hooks. These concepts are essential for building scalable and maintainable React applications.

Task 22: Redux Essentials – Overview and Concepts

Summary

In this task, I learned the core concepts of Redux and how it manages application state using a predictable data flow. I understood that Redux stores all shared application data in a single store, making it easier to manage state across multiple components. This approach helps maintain consistency and simplifies debugging in larger applications.

I also learned about the main building blocks of Redux, including the Store, State, Actions, Reducers, and Dispatch. An action describes what happened, a reducer determines how the state should change, and the store holds the updated state. Components dispatch actions to request state updates and read the latest state from the Redux store whenever needed.

Additionally, I understood why Redux is useful for applications with complex or shared state. It provides a predictable state management pattern, improves code organization, and makes applications easier to test, debug, and maintain. This task gave me a strong foundation for understanding how Redux works before implementing it in React projects using Redux Toolkit.

Learning Outcome

After completing this task, I understood the fundamental concepts of Redux, including the Store, State, Actions, Reducers, and Dispatch process. I also learned how Redux provides predictable state management and serves as the foundation for building scalable React applications using Redux Toolkit.

⇒ GITHUB REPO LINK:

https://github.com/yashrathod910/15D_Internship/tree/main/DAY13